

sol3_research_deep_agents. White paper

A question in, a cited brief out. A topic in, an evidence-backed white paper out.

LangChain Deep Agents. Two entry points. One role table.

PR #156 grew the Saturday brief into a nine-stage report pipeline. Saturday Lab 3 still fills two functions. This folder writes `whitepaper.md`.

Spillwave Solutions | spillwave.com

Two entry points. Same folder.

```
python3 loop.py --question "sqlalchemy nullable datetime column"  
python3 loop.py --paper --topic "context management in multi-agent research loops"
```

The brief is the small version of the paper. Both plan questions, search through one tool boundary, and refuse to ship anything uncited. The paper adds corroboration, figures, and a publish step.

What you will walk through

FILE	HOLDS
loop.py	both entry points, and which cast to print
roleplan.py	research (four roles) and paper (seven)
paper.py	nine stages, three exits
stages.py	each stage's gate
evidence.py	SourceDocument, Claim, Finding
paper_check.py	hard gates on a finished paper
publish.py	secret gist, one id per paper forever
roles.py	three fencing layers

Why this folder exists

LangChain's own Deep Agents quickstart is a research agent. This folder is that shape, pointed at our tool boundary, then grown until it produces something you would hand to a colleague.

A role cannot do a job it was not given a tool for. Nothing about grounding, cost, or stopping is left to a model's own judgment.

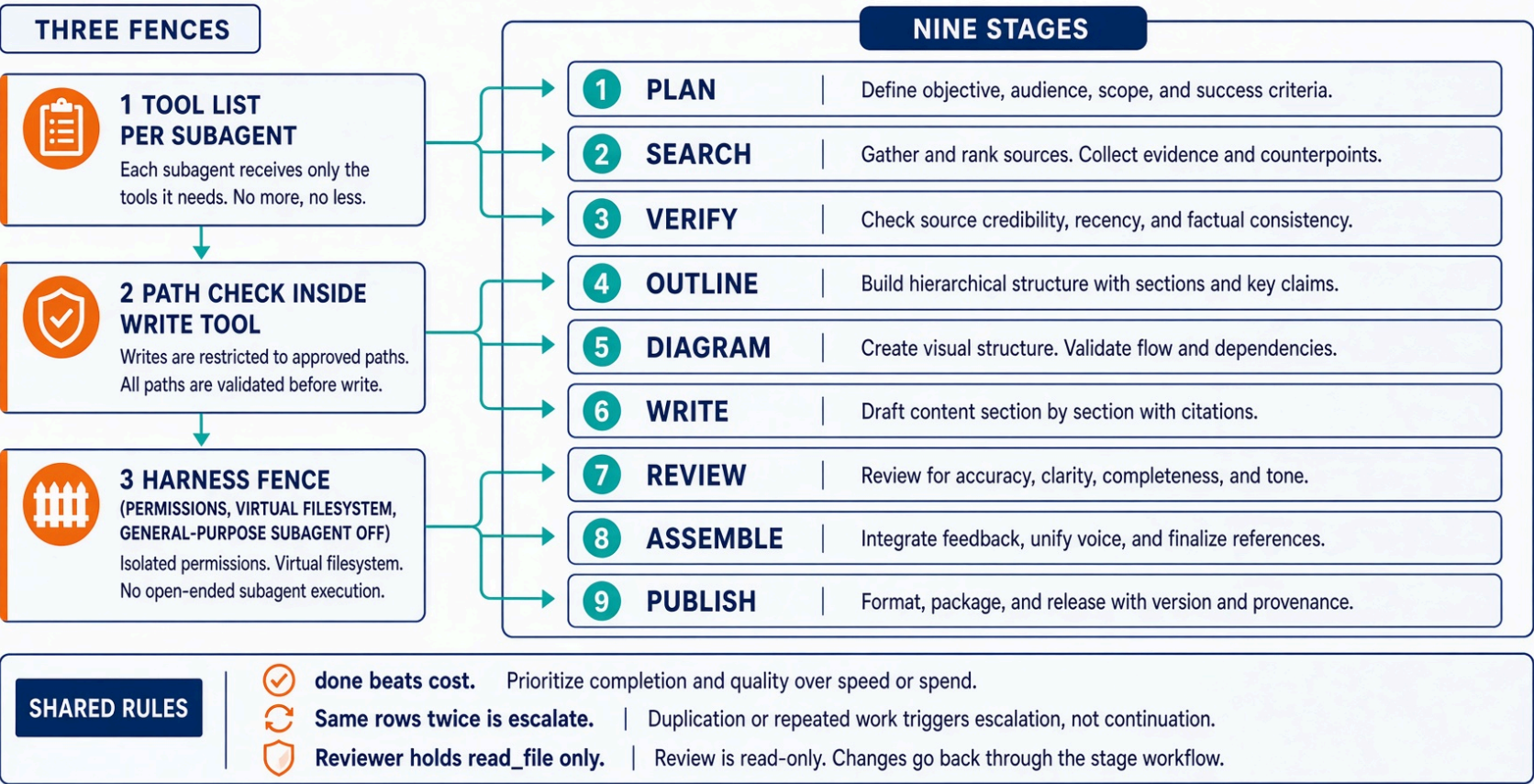
Spillwave Solutions | spillwave.com

Learning objectives

- Print the paper cast: reviewer writes `no`
- Name the three fences Deep Agents actually uses
- Walk nine stages and say which two call no model
- Prefer `done` over a spent budget
- Keep the Saturday brief working (`--question`)
- Refuse `task publish` when `gates.json` is red

Starting architecture

Deep Agents white paper. Nine stages. Three fences.



The paper cast

role	writes	scope
orchestrator	no	nothing
planner	yes	plan.json
researcher	no	nothing
verifier	yes	evidence/** denied: paper/**
diagrammer	yes	diagrams/*.mmd, *.puml
writer	yes	brief.md, paper/**, work/research/**
reviewer	no	nothing

`task table (adds --paper)`. If the reviewer prints `yes`, the translation is wrong and the tests say so.

A role that would hold the same tools as its neighbour is not a role. It is a prompt, and it belongs in a skill.

Three fences. People skip the third.

1. **A tool list per subagent.** The reviewer is handed `read_file` and nothing else. "Do not edit the paper" is not an instruction it can reinterpret.
2. **A path check inside the write tool.** Writer may write `paper/**`. Verifier may write `evidence/**`. Neither can reach the other. Refusal is a sentence, not a traceback: a raw traceback starts a retry loop.
3. **A fence around the harness.** `permissions=`, hide built-in write tools from the orchestrator, `FilesystemBackend(virtual_mode=True)` SO `..` cannot walk out, general-purpose subagent **off**.

Layer 3 is the one people skip. The default general-purpose subagent ships with the harness filesystem tools. Leaving it enabled is how a carefully scoped agent writes anywhere it likes.

Nine stages. Two of them are Python.

#	STAGE	MODEL?	GATE
1	plan	yes	3 to 8 questions, each with a check
2	search	yes	every claim names a source that resolves
3	verify	yes	every important claim has a decided truth state
4	outline	yes	every body section names claim ids that exist
5	diagram	yes	at most 12 nodes, every figure has alt text
6	write	yes	a section cites only the numbers its claims support
7	review	yes	every rubric row passes
8	assemble	no	every hard gate in <code>paper_check</code>
9	publish	no	the gates passed. Opt-in only

Assembling markdown and pushing a gist are things a program does correctly every time. Handing either to a model buys a new failure mode.

Three exits. Done first.

Checked before every stage:

EXIT	WHEN
done	stage 8 finished and every hard gate is green
cost	the money budget is spent
max turns	a stage exhausted its retries

Done beats a spent budget. A run that finished and then noticed it was over its cap did finish. Reporting that as a cost failure discards the paper.

Inside a stage, `gates.decide` short-circuits the same rows failing twice.

One boundary. Four backends.

The loop never learns which one answered.

BACKEND	WHEN
perplexity	PERPLEXITY_API_KEY. MCP first, then REST
context7	ctx7 on PATH, or the MCP server. Verification only
websearch	coding agent through an inbox file
fixture	always. A recording, for a room with no network

task paper is fixture. task live is auto.

Saturday brief is still here

```
def plan_questions(question: str) -> list[str]:  
    return [question, f"{question} common mistake", f"{question} how to verify"]  
  
def check_brief(body, sources):  
    return brief.check(body, sources)
```

Four arithmetic rows: `has_sources`, `grounded`, `cited`, `style`.

```
task brief -- --question "sqlalchemy nullable datetime column"
```

That is the Lab 3 answer. The paper is the take-home grown from it.

Evidence records outlive the paper

```
work/paper/<slug>/
  whitepaper.md
  plan.json
  outline.json
  evidence/      SourceDocument, Claim, Finding
  diagrams/      mermaid and plantuml sources
  figures/       SVG, polished PNG, sidecar
  gates.json
  .paper-state.json  resume reads this
```

Same record shape as `loop_eng_2nd_brain/knowledge/research/`. A run can be ingested later without a conversion step. The claims outlive the paper, which is the point of writing them down separately.

Hard gates on the finished paper

`paper_check.py`. No model. A failing hard gate blocks publish.

Required sections: abstract, introduction, references. Recommended: limitations.

Also: min 400 words, figures need alt text, diagram source must not survive into the body, citation arithmetic reused from `brief.py`.

`signature()` is the sorted names of **hard** failures. A soft warning that keeps firing would look like a stall and escalate a run that is converging.

Publish. Four load-bearing rules.

1. **One gist per paper, forever.** Id in `gist-ids.tsv`. Refresh, do not mint a new gist per run.
2. **Secret, never public.** Unlisted, not private. The URL is the credential.
3. **Figures inline.** A gist is flat. Copy figures to the root, rewrite image links to raw URLs.
4. **The gate comes first.** `push` refuses when `gates.json` is red.

`task paper` never publishes. `task publish` does.

```
task publish -- --slug ... --dry-run --out /tmp/staged
```

Resume and fixtures

A resumed live run skips every finished stage and carries the cost forward.

The fixture runner keys replies by a phrase in the prompt, not by position.

A positional queue restarts at zero when the run resumes at stage six and hands the writer the outline reply.

A list of replies repeats the last entry. That repeat is deliberate: a retry gets the same answer, which is the stable failure `gates.decide` exists to catch, so the offline run can demonstrate escalate without anybody faking it.

Commands

```
cd solutions/sol3_research_deep_agents
python3 -m venv .venv && source .venv/bin/activate && pip install pytest
task table          # paper cast, no SDK
task checks         # evidence, paper_check, diagrams, publish, mcp_tools
task test           # 198 tests. No SDK, no key, no network
task paper          # nine stages, fixture backend
task setup          # only for a live run
task live TOPIC="context management in multi-agent research loops"
task publish -- --slug context-management-in-multi-agent-research-loops
```

Standalone. Tests pin it.

tests/test_standalone.py fails if this folder imports a shared engine.

Other pins:

TEST	ASSERTS
role table	reviewer holds no write
stages	plan 3 to 8 questions; outline binds real claim ids
paper_check	missing abstract fails; source syntax in body fails
publish	red gates.json refuses
research	[9] is ungrounded; budget hard cap raises

Troubleshooting

SYMPTOM	CAUSE	FIX
Reviewer writes <code>yes</code>	tool list leaked	<code>read_file</code> only
Agent writes anywhere	general-purpose subagent left on	turn it off
Traceback retry storm	write tool raised	return a refusal sentence
Resume hands wrong reply	positional fixture	key by prompt phrase
New gist every run	ignored <code>gist-ids.tsv</code>	one id per slug
Published a red paper	skipped <code>gates.json</code>	<code>push</code> must refuse
Saturday brief missing	ran <code>--paper</code> by accident	<code>--question</code> is the lab

Validation checklist

- ☐ `task table:reviewer no`
- ☐ `task test is198, no SDK`
- ☐ `task paper writes whitepaper.md plus evidence/`
- ☐ Stage 8 and 9 call no model
- ☐ Done beats cost
- ☐ `task publish -- --dry-run restages figures`
- ☐ `tests/test_standalone.py` still passes

Spillwave Solutions | spillwave.com

Recap

What we built. A brief for Saturday. A paper for take-home. Same exits.

Takeaways

1. Two entry points. One table.
2. Three fences. The third is the one people skip.
3. Assemble and publish are Python.
4. Done beats cost.
5. Claims outlive the paper.

Closing line. A role that holds the same tools as its neighbour is a prompt.